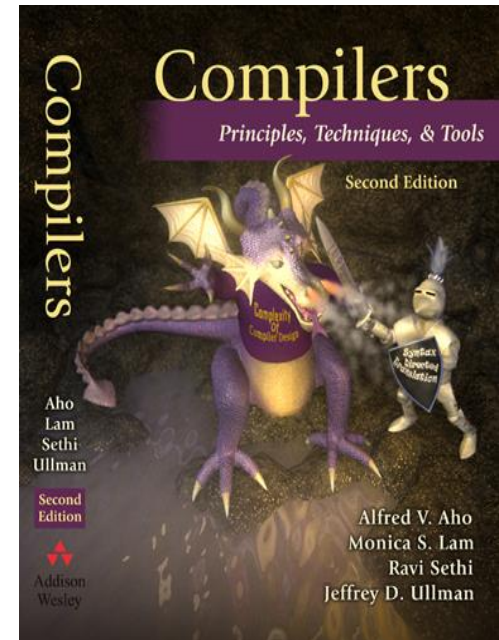


Text

- The Purple Dragon Book
- Aho, Lam, Sethi & Ullman
- Not required
 - But a useful reference



History of Compilers

- 1954 IBM develops the 704
 - Successor to the 701
- Problem
 - Software costs exceeded hardware costs!
- All programming done in assembly



The Solution

- “Speedcoding”
 - an early example of an interpreter
 - developed in 1953 by John Backus
 - much faster way of developing programs
 - programs were 10-20 times slower than hand-written assembly
 - needed 300 bytes = 30% machine memory



John Backus

FORTRAN I

- FORMula TRANslation Project
- FORTRAN ran from 1954 To 1957
- By 1958, over 50 percent of all of programs were in FORTRAN



John Backus

FORTRAN I

- The first compiler
 - Huge impact on computer science
- Led to an enormous body of theoretical work
- Modern compilers preserve the outlines of FORTRAN I

The Structure of Fortran Compiler

1. Lexical Analysis
2. Parsing
3. Semantic Analysis
4. Optimization
5. Code Generation

The first 3, at least, can be understood by analogy to how humans comprehend English.

Lexical Analysis

- First step: recognize words.
 - Smallest unit above letters

This is a sentence.

More Lexical Analysis

- Lexical analysis is not trivial. Consider:

ist his ase nte nce

And More Lexical Analysis

- Lexical analyzer divides program text into “words” or “tokens”

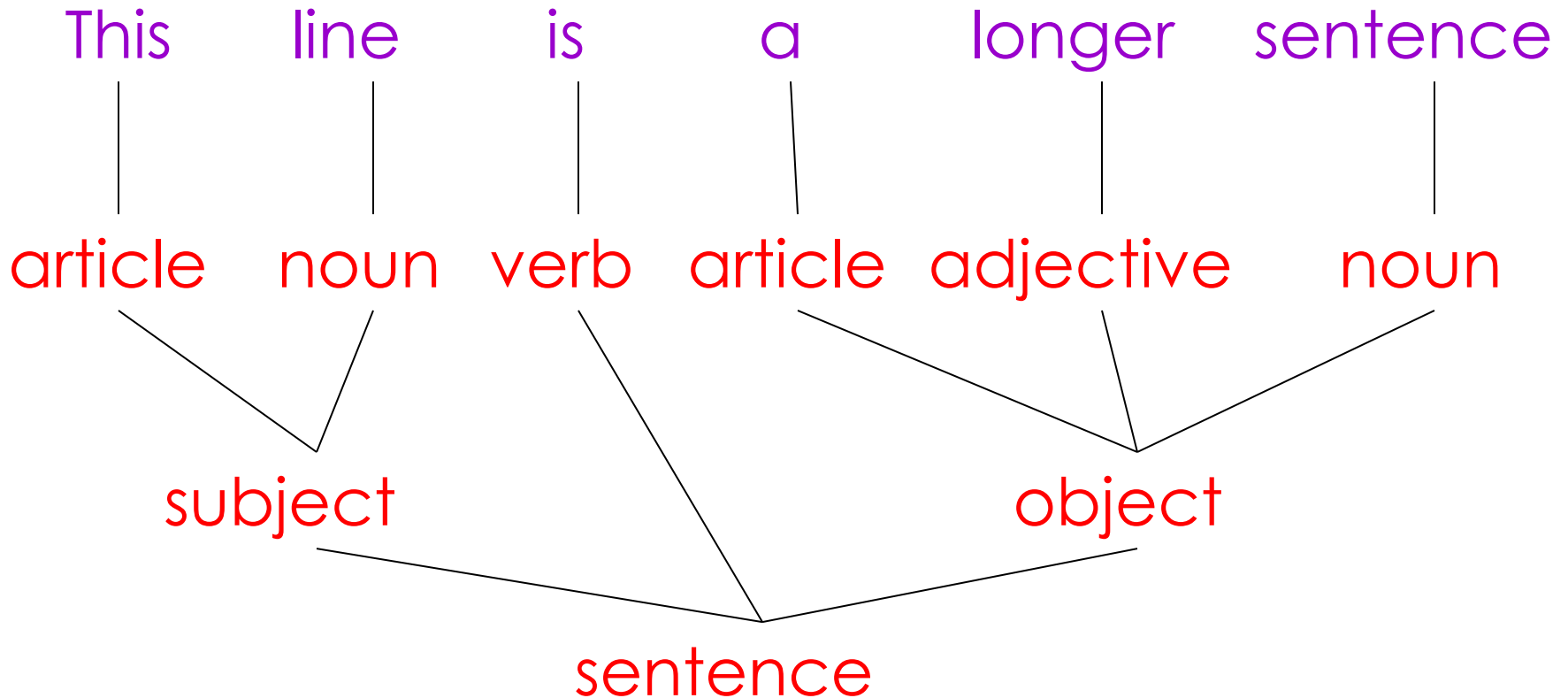
If x == y then z = 1; else z = 2;

- Units:
 - Keywords { if, then, else }
 - Identifiers { x, y, z }
 - Numbers { 1, 2 }
 - Operators { ==, = }
 - Separators { blanks, ; }

Parsing

- Once words are understood, the next step is to understand sentence structure
- Parsing = Diagramming Sentences
 - The diagram is a tree

Diagramming a Sentence

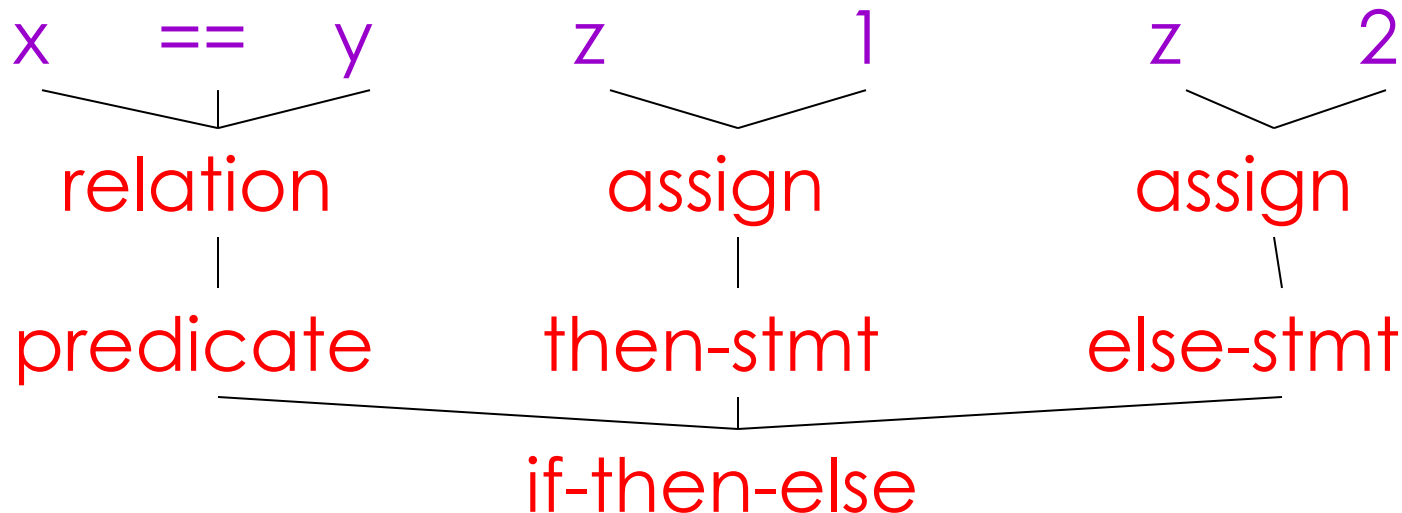


Parsing Programs

- Parsing program expressions is the same
- Consider:

If $x == y$ then $z = 1$; else $z = 2$;

- Diagrammed:



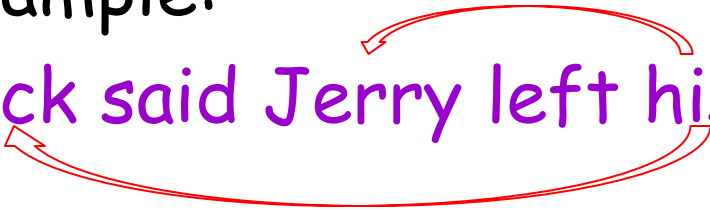
Semantic Analysis

- Once sentence structure is understood, we can try to understand “meaning”
 - But meaning is too hard for compilers
- Compilers perform limited semantic analysis to catch inconsistencies

Semantic Analysis in English

- Example:

Jack said Jerry left his assignment at home.



What does “his” refer to? Jack or Jerry?

- Even worse:

Jack said Jack left his assignment at home?

How many Jacks are there? (1, 2 , or 3)

Which one left the assignment?

Semantic Analysis in Programming

- Programming languages define strict rules to avoid such ambiguities
- This C++ code prints “4”; the inner definition is used

```
{  
    int Jack = 3;  
    {  
        int Jack = 4;  
        cout << Jack;  
    }  
}
```

More Semantic Analysis

- Compilers perform many semantic checks besides variable bindings

- Example:

Jack left her homework at home.

- A “type mismatch” between her and Jack; we know they are different people
 - Presumably Jack is male

Optimization

- No strong counterpart in English,
 - but a little bit like editing
 - but akin to editing
- Automatically modify programs so that they
 - Run faster
 - Use less memory
- Your project has no optimization component :D

Optimization Example

$X = Y * 0$ is the same as $X = 0$

NOT ALWAYS CORRECT

NaN

$\text{NaN} * 0 = \text{NaN}$

Code Generation

- Produces assembly code (usually)
- A translation into another language
 - Analogous to human translation

Compilers Today

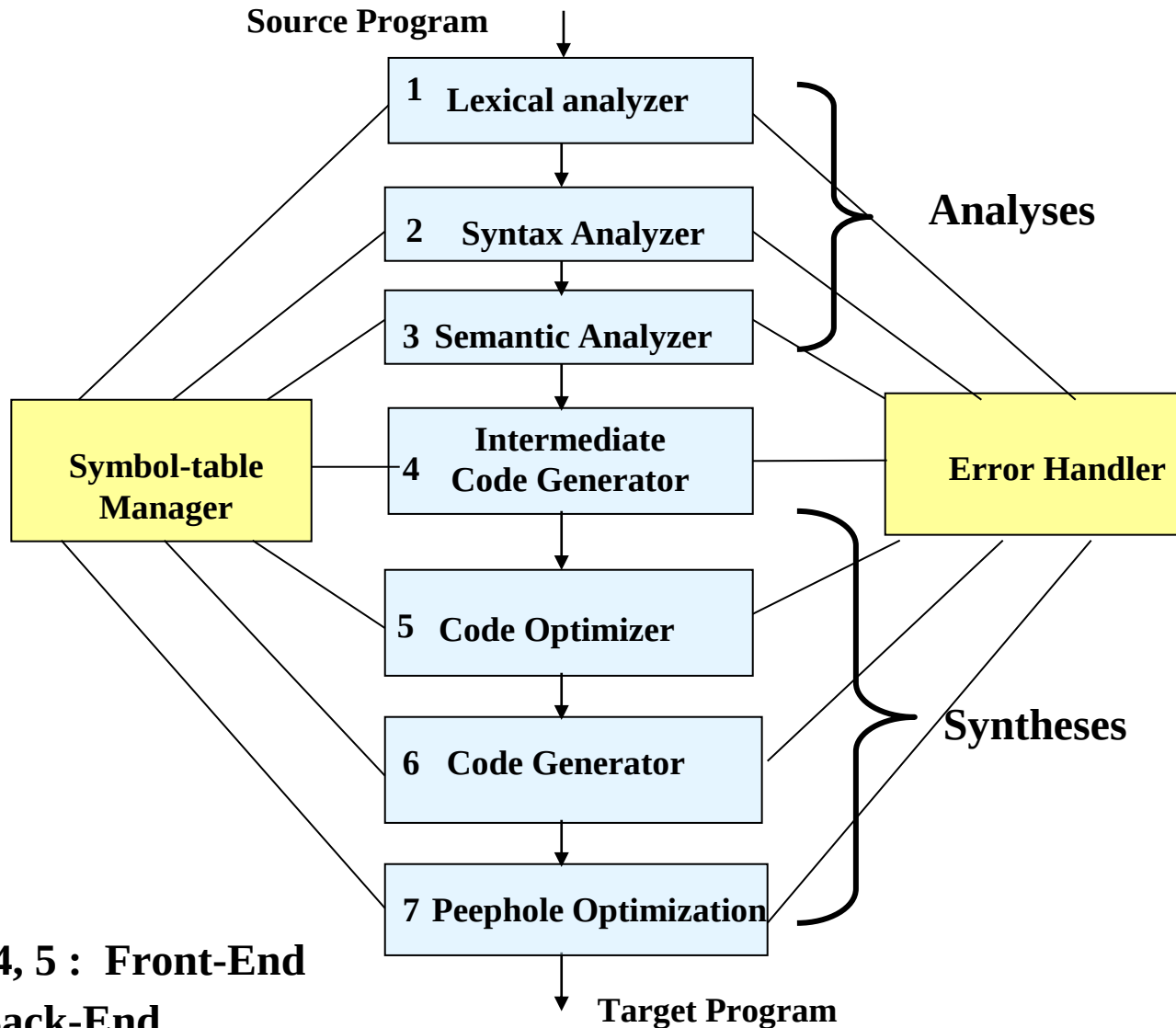
- The overall structure of almost every compiler adheres to our outline
- The proportions have changed since FORTRAN
 - Early: lexing, parsing most complex, expensive



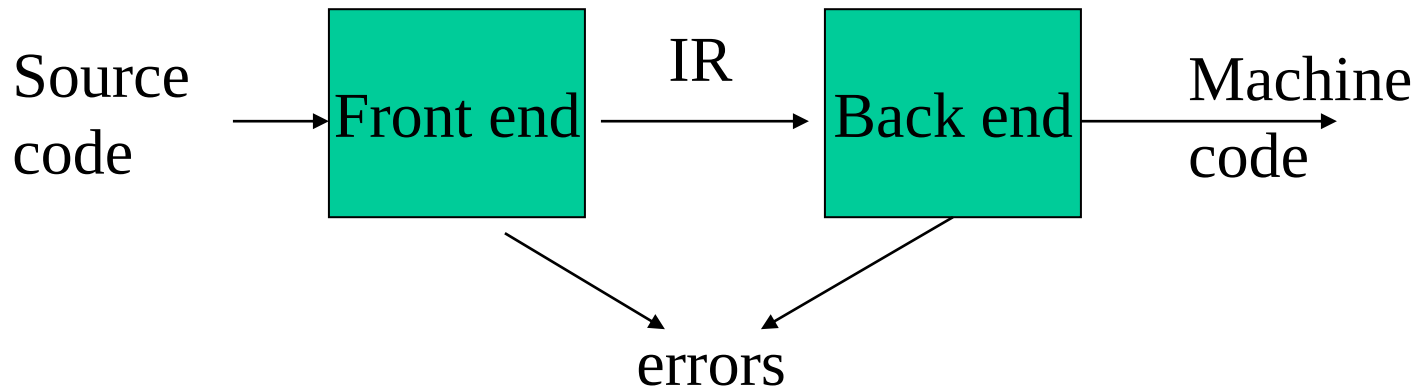
- Today: optimization dominates all other phases, lexing and parsing are cheap



Compiler Front-end and Back-end

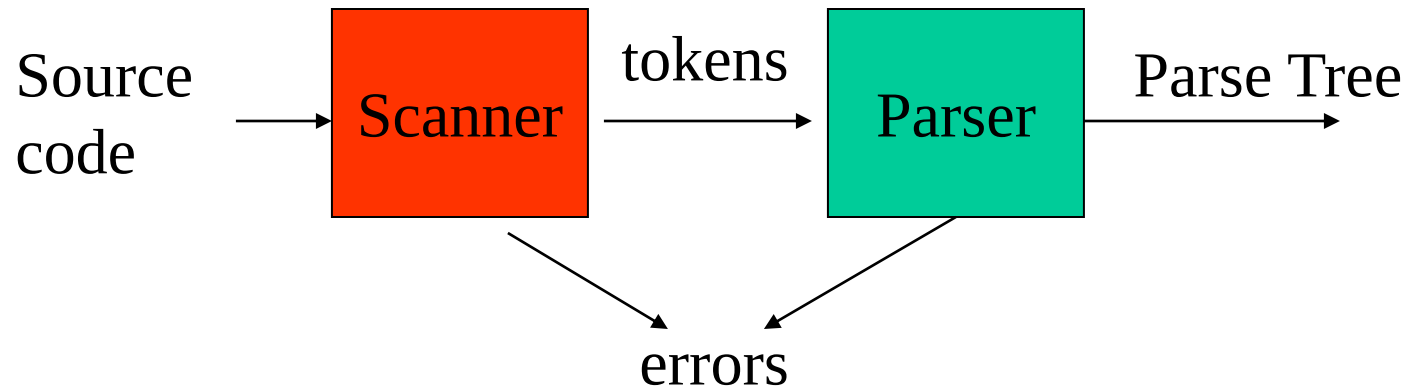


Front-End



- Front end maps source code into an IR representation
- Back end maps IR onto machine code
- Simplifies retargeting

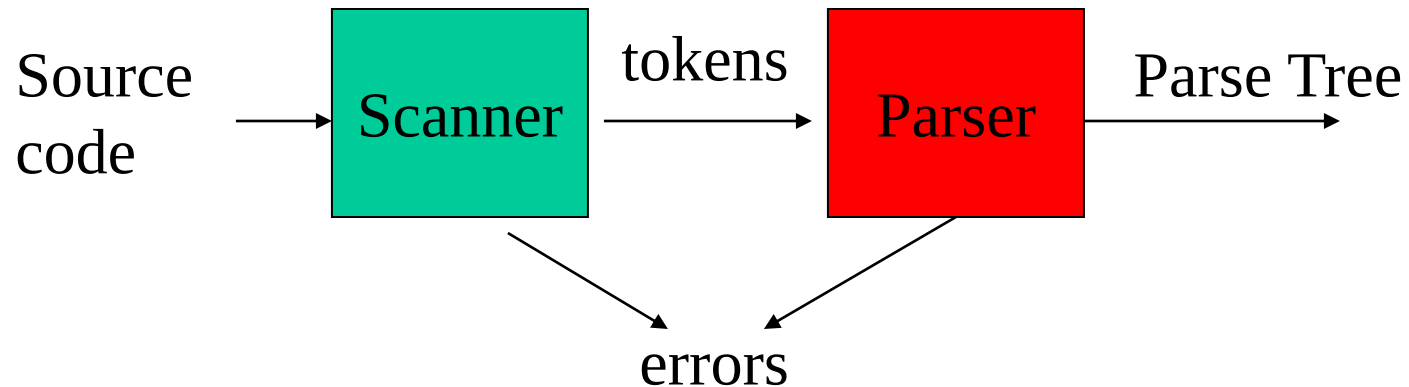
Front-End (Cont.)



Scanner:

- Maps characters into tokens - the basic unit of syntax
 - $x = x + y$ becomes $\langle \text{id}, x \rangle \langle =, \rangle \langle \text{id}, x \rangle \langle +, \rangle \langle \text{id}, y \rangle$
- Eliminate white space (tabs, blanks, comments)

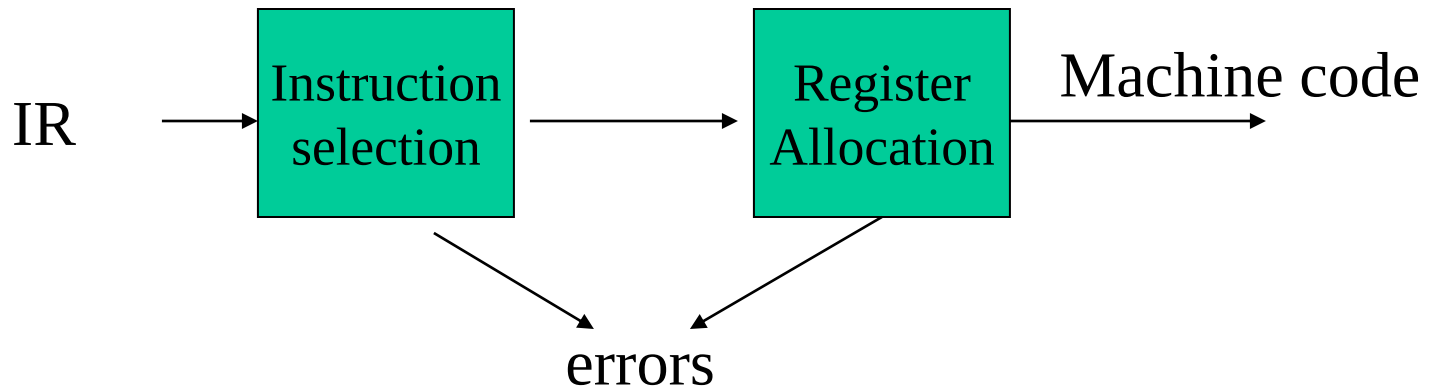
Front-End (Cont.)



Parser:

- Recognize context-free syntax
- Guide context-sensitive analysis
- Produce meaningful error messages

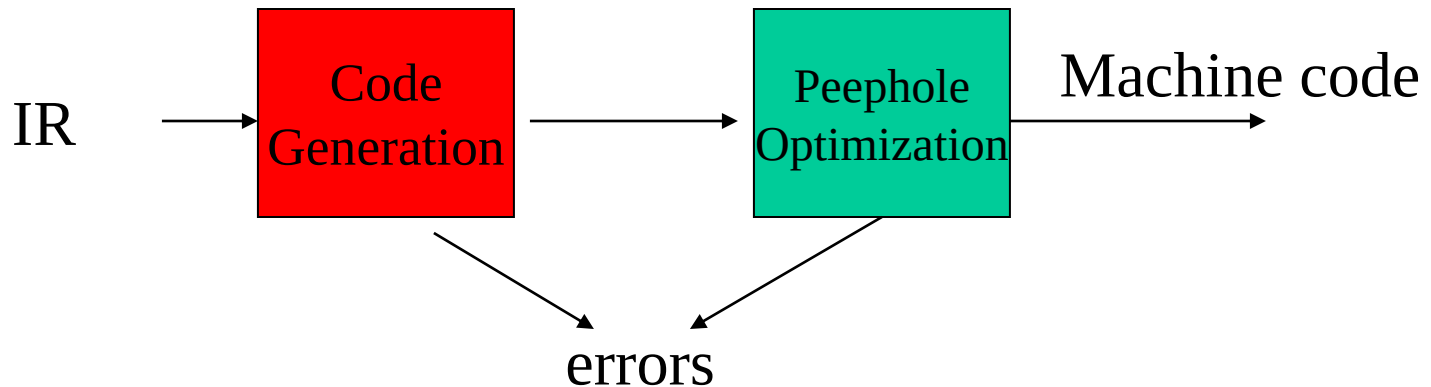
Back-End



Back-End:

- Translate IR into machine code
- Choose instructions for each IR operation
- Decide what to keep in registers at each point

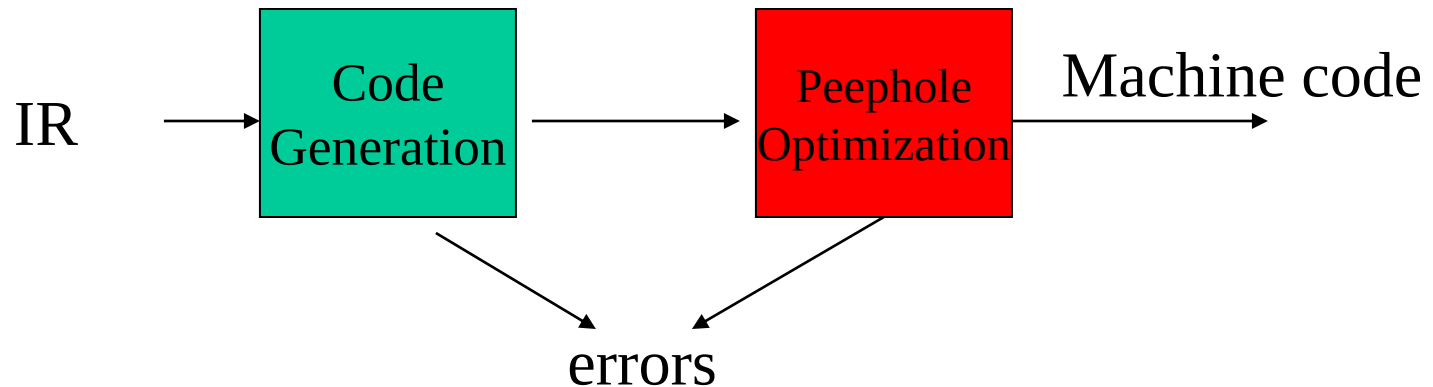
Two Main Components of Back-End



Code Generator:

- Produce compact fast code
- Use available addressing modes

Back-End (Cont.)



Peephole Optimization:

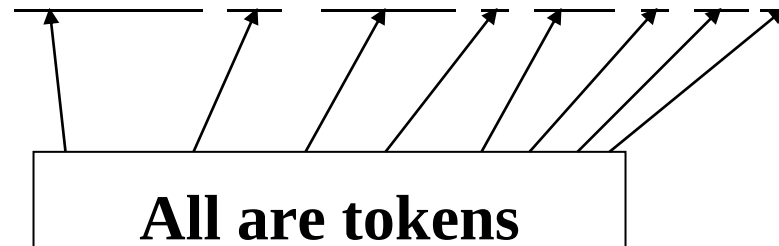
- Limited resources
- Optimal allocation is difficult

Phase 1. Lexical Analysis

Easiest Analysis - Identify tokens which are the basic building blocks

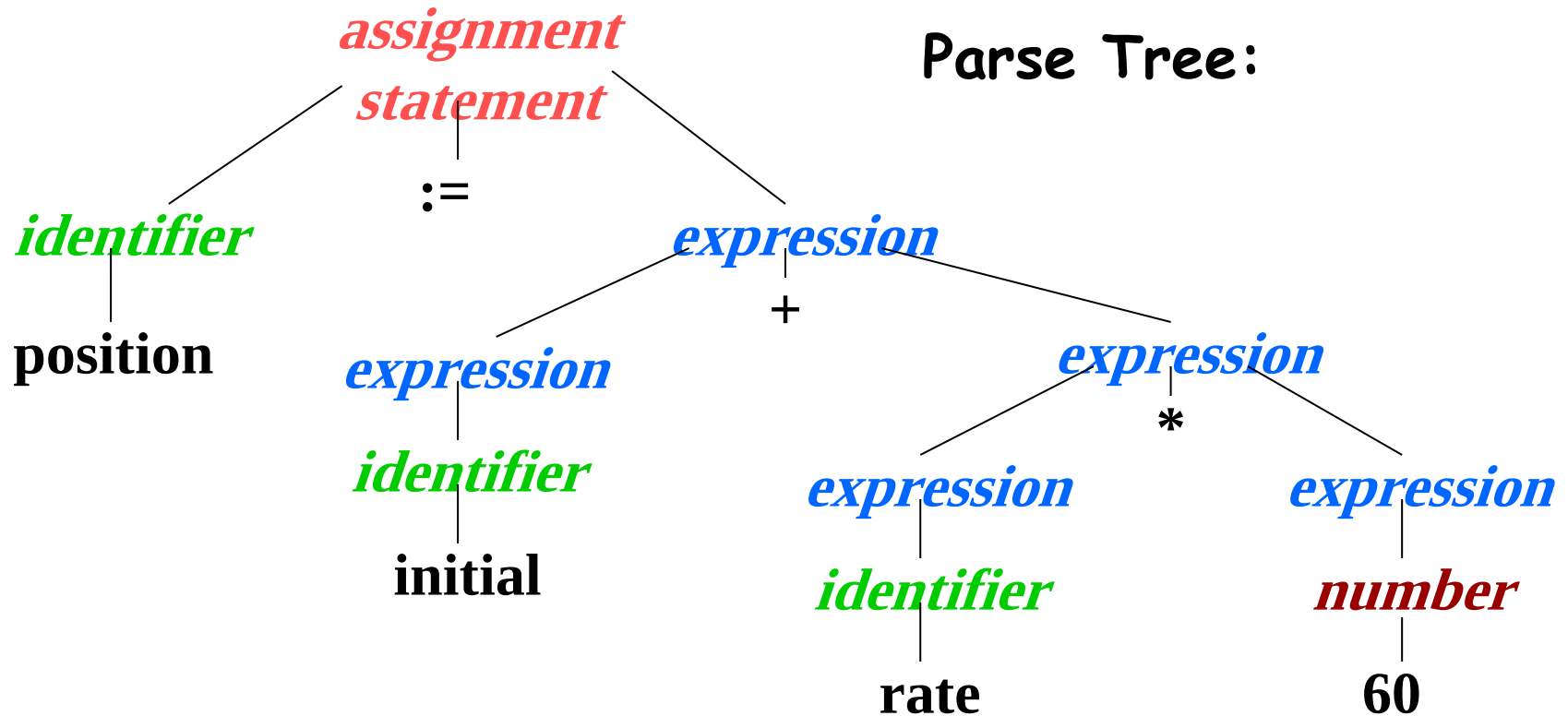
For

Example: **Position := initial + rate * 60 ;**



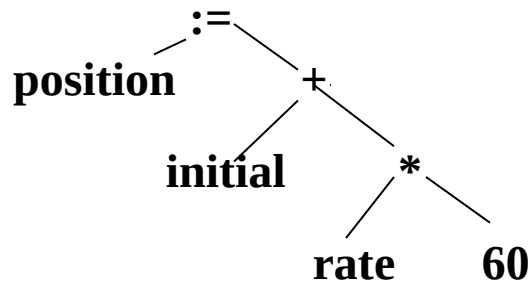
Blanks, Line breaks, etc. are scanned out

Phase 2. Syntax Analysis or Parsing

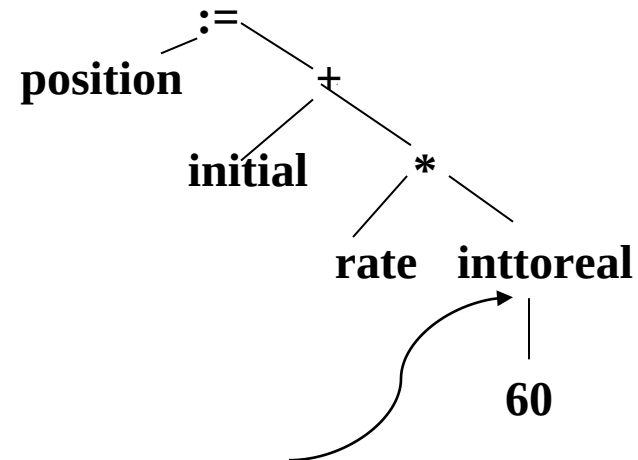


Nodes of tree are constructed using a Grammar for the source language

-
- Finds Semantic Errors



Syntax Tree



Conversion Action

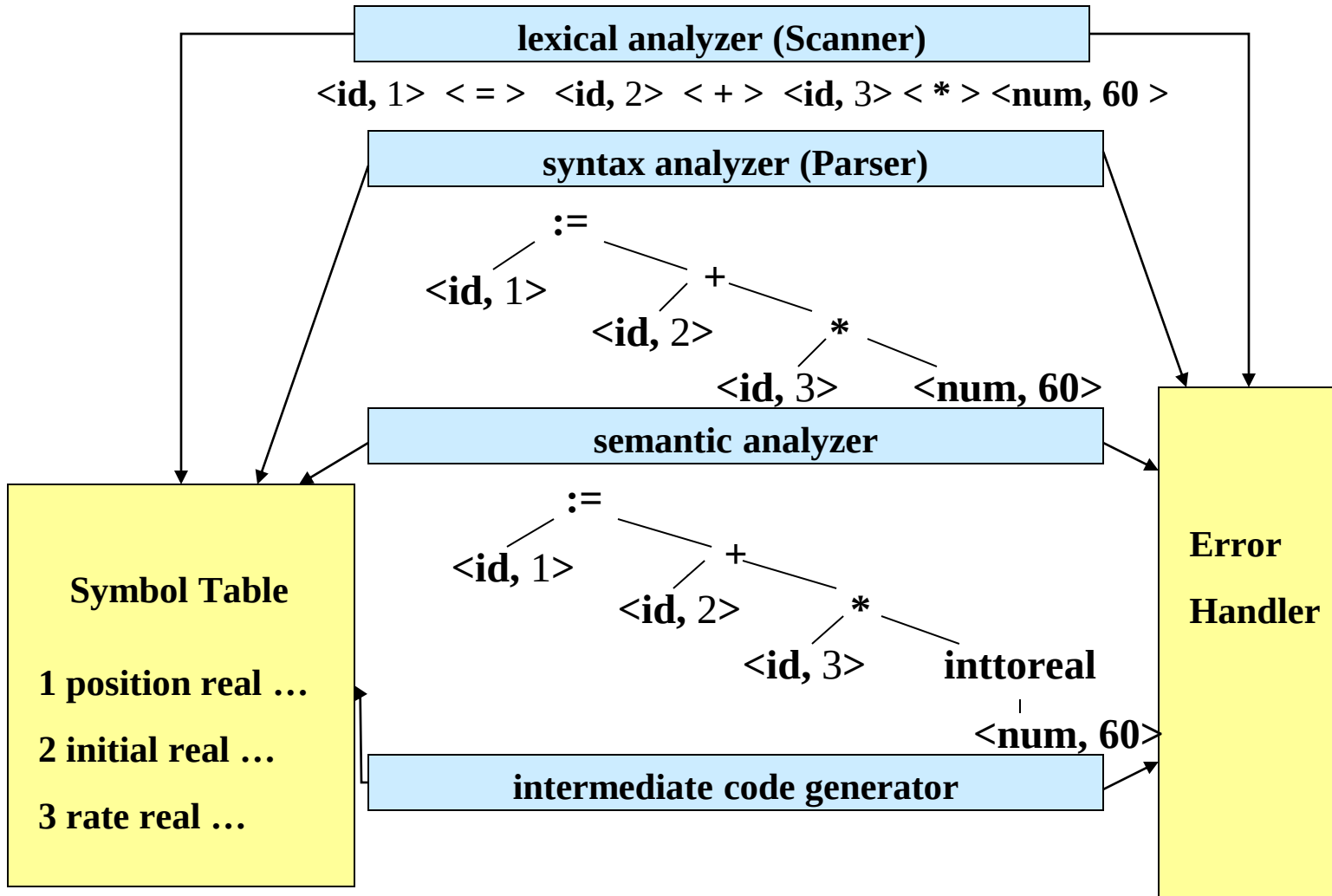
- One of the Most Important Activities in This Phase:
- Type Checking - Legality of Operands

Supporting Phases

- Symbol table creation / maintenance
 - Contains info (address, type, scope, args) on certain Tokens, typically identifiers
 - Data structure created/initialized during lexical analysis; and updated during later analysis & synthesis
- Error handling
 - Detection of different errors which correspond to all phases; and deciding what happens when an error is found

An example of the Entire Process

`position = initial + rate * 60`



An example of the Entire Process

